

MEMORIA TÉCNICA- PERCEPTRÓN DE ROSENBLATT

1. Introducción y Dataset

En esta práctica se aborda el reto de construir un modelo de aprendizaje automático desde una perspectiva de "Caja Blanca" (White Box). En lugar de utilizar librerías de alto nivel que ocultan la lógica, se ha implementado el Perceptrón de Rosenblatt desde cero. El objetivo es comprender cómo una neurona artificial puede aprender mediante el ajuste iterativo de pesos y un sesgo (bias).

Para esta práctica se ha seleccionado un dataset de Clasificación Binaria que analiza las preferencias de vacaciones de diferentes personas.

- **Dataset:** *Mountains vs. Beaches Preference Dataset*.
- **Enlace:** [Kaggle - Mountains vs Beaches Preference](#)
- **Objetivo:** Predecir si un usuario prefiere la **Montaña (1)** o la **Playa (0)** basándose en datos demográficos y geográficos.
- **Descripción:** Se trata de una colección de **52.444** instancias con variables como edad, ingresos y distancias geográficas.

2. Preparación y tratamiento de los datos

Siguiendo los objetivos de la práctica, se ha realizado un tratamiento de datos manual sin usar librerías de aprendizaje automático:

1. **Carga** del dataset desde un archivo CSV utilizando la librería pandas.
2. **Selección de Variables:** Para simplificar el modelo en esta primera etapa, se han seleccionado 4 variables numéricas clave: Age (Edad), Travel_Frequency (Frecuencia de viaje), Proximity_to_Mountains (Distancia a montañas) y Proximity_to_Beaches (Distancia a playas).
3. **División (Split):** Se dividió el dataset manualmente: 80% Entrenamiento (aprendizaje) y 20% Test (validación final).
4. **Escalado estándar:** Se implementó una normalización manual ($X = (X - \mu) / \sigma$) para situar todas las variables en una escala similar y evitar que las variables con rangos grandes dominen sobre otras.
5. **Formato:** Conversión de los datos a matrices de NumPy para operar con la "unión sumadora".

3. Desarrollo del Modelo

Se ha implementado un Perceptrón de Rosenblatt desde cero que recrea la lógica interna de una neurona artificial:

- **Unión Sumadora:** Se calcula mediante el producto escalar de las entradas por los pesos más el sesgo ($z = \sum x_i \cdot w_i + b$).
- **Función de Activación:** Se ha utilizado una función escalón (Unit Step). Si la suma es ≥ 0 la salida es 1, de lo contrario es 0.
- **Regla de Aprendizaje:** En cada época, la neurona compara su predicción con el valor real. Si falla, ajusta sus pesos y el sesgo:
- **Cambio en el peso:** $\Delta w = \eta * (y - \hat{y}) * x$
- **Actualización del peso:** $w = w + \Delta w$
- **Actualización del sesgo:** $b = b + \eta * (y - \hat{y})$

Donde:

- **Δw (Delta w):** Es el ajuste o cambio que le vamos a hacer al peso.
- **η (Eta):** Es la tasa de aprendizaje (el 0.1 que pusimos en el código).
- **y :** Es el valor real (lo que dice el dataset). ○ **$\hat{y}$:** Es la predicción que hizo la neurona. ○ **x :** Es el dato de entrada (ej. la edad o la distancia).

4. Traza del Ajuste y Resultados

El modelo se configuró con una tasa de aprendizaje (η) de 0.1 y un total de 10 épocas.

Decisión sobre el tiempo de cómputo: Aunque el valor por defecto en el código era de 50 épocas, se ajustó a 10 épocas para la ejecución final. Dado que el dataset es grande (más de 50.000 registros), aumentar el número de épocas incrementaría proporcionalmente el tiempo de espera y el esfuerzo del procesador. Se comprobó que con 10 iteraciones completas el modelo ya alcanzaba una estabilidad óptima.

Resultados obtenidos en consola:

```
C:\Users\IVANA\Desktop\Perceptron simple>python perceptron.py

Entrenando la neurona...
Gráfica generada y guardada como 'grafica_entrenamiento.png'

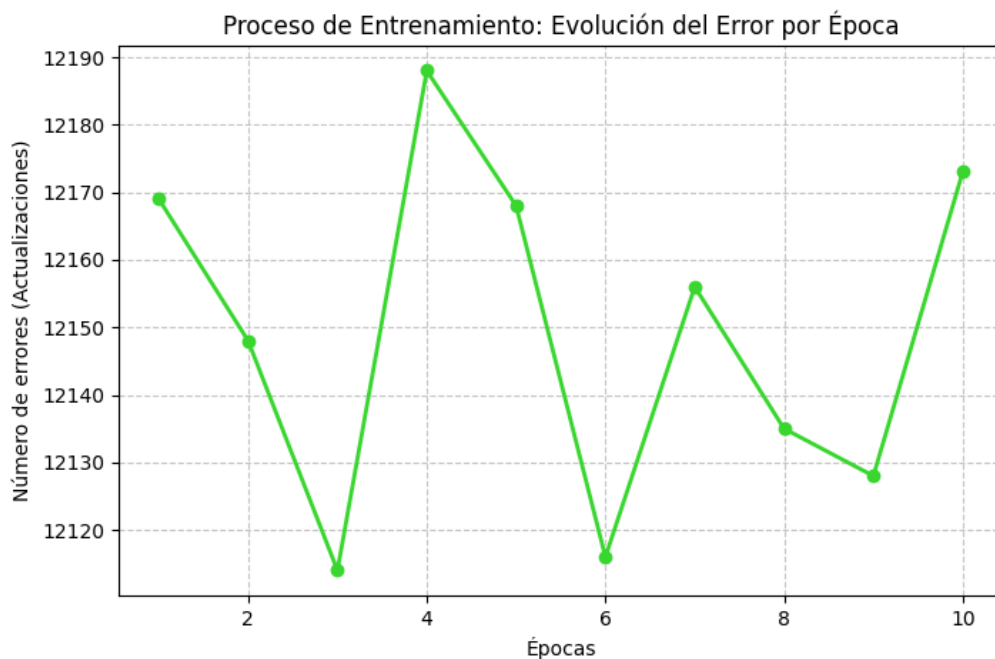
=====
RESULTADOS
=====

La neurona ha acertado el 74.66% de los casos.
Pesos finales (importancia): [ 0.06354216 -0.05327369 -0.06952168  0.2309113 ]
Sesgo final (bias): -0.30000000000000004

C:\Users\IVANA\Desktop\Perceptron simple>
```

- **Precisión (Accuracy) en Test: 74.66%.**
- **Pesos finales: [0.0635, -0.0532, -0.0695, 0.2309]**
- **Sesgo (Bias) final: -0.30**

Análisis de los pesos: Se observa que la variable con mayor peso en valor absoluto es la proximidad a la playa (0.23). Esto indica que la distancia geográfica es el factor que más influye en la decisión de la neurona. El sesgo negativo desplaza la frontera de decisión, compensando la distribución de las entradas y evitando una clasificación excesivamente optimista hacia la clase positiva.



La gráfica de errores por época muestra que, aunque el número de errores no disminuye de forma continua, el modelo llega a un comportamiento estable, lo que indica que los pesos han aprendido una solución adecuada para el problema.

5. Conclusión

La realización de esta práctica ha permitido aplicar los conceptos teóricos del Perceptrón de Rosenblatt en un entorno real. Al construir el modelo desde cero, se ha logrado pasar de ver el algoritmo como una herramienta de tipo "Black Box" a comprenderlo como una "White Box", donde es posible auditar cómo cada peso y el sesgo influyen directamente en la clasificación final.

Obtener una precisión del **74.66%** en este primer ejercicio es un resultado satisfactorio. Demuestra que, incluso sin aplicar técnicas avanzadas de optimización o modelos más complejos, una sola neurona artificial es capaz de identificar patrones lógicos en datasets con miles de registros de forma autónoma.